

Asignatura Generalización de Código de Automatización en Desarrollo de Software con IA	Datos del alumno Apellidos: Chavarria González Nombre: Marina	Fecha Junio 2025
--	--	----------------------------

Actividad 2

Generación de entornos de pruebas y casos de test asistidos por IA

1. Análisis de la aplicación existente

La aplicación PlantCare, desarrollada en la Actividad 1, es una aplicación web 100% cliente que gestiona el riego de plantas usando Vanilla JS y localStorage. Para definir el alcance de las pruebas se solicitó al asistente de IA que identificara las funcionalidades críticas.

Prompt utilizado:

```
Analiza el siguiente proyecto web (PlantCare) e identifica las funcionalidades críticas que deben cubrirse con pruebas automatizadas. Describe también qué tipo de pruebas (unitarias, integración, E2E) serían más adecuadas para cada una.
```

Respuesta del asistente (resumen):

Funcionalidad	Tipo de prueba	Prioridad
Añadir planta (validación de campos)	Unitaria / Integración	Alta
Eliminar planta con confirmación	Integración / E2E	Alta
Editar planta existente	Integración	Alta
Cálculo del estado de riego	Unitaria	Alta
Barra de progreso de riego	Unitaria	Media
Persistencia en localStorage	Integración	Alta
Panel de resumen (hero banner)	Integración / E2E	Media
Navegación entre vistas (tarjetas/calendario)	E2E	Media
Calendario mensual	Integración / E2E	Baja

2. Configuración del entorno de pruebas

Dado que PlantCare es una aplicación Vanilla JS sin módulos ni bundler, se optó por una estrategia doble:

- Jest + jsdom para pruebas unitarias y de integración de la lógica JavaScript.
- Playwright para pruebas de interfaz (E2E).

El siguiente prompt fue enviado al asistente para generar la configuración inicial:

Prompt utilizado:

```
Genera la configuración completa para un entorno de pruebas sobre una aplicación Vanilla JS que usa localStorage. Necesito: 1. package.json con scripts de test 2. jest.config.js con jsdom como entorno 3. Un setup global que mockee localStorage 4. Instrucciones para instalar todo con npm
```

Configuración generada (package.json — fragmento):

```
{  "scripts": {    "test": "jest --coverage",    "test:e2e": "playwright test"  },  "devDependencies": {    "jest": "^29.0.0",    "@jest/globals": "^29.0.0",    "jest-environment-jsdom": "^29.0.0",    "@playwright/test": "^1.40.0"  } }
```

jest.config.js generado:

```
module.exports = {  testEnvironment: "jsdom",  setupFilesAfterFramework: ["./tests/setup.js"],  collectCoverageFrom: ["src/**/*.js"] };
```

setup.js — mock de localStorage:

```
const localStorageMock = () => {  let store = {};  return {    getItem: (k) => store[k] ?? null,    setItem: (k, v) => { store[k] = String(v); },    removeItem: (k) => { delete store[k]; },    clear: () => { store = {}; },  }; }(); Object.defineProperty(window, 'localStorage', { value: localStorageMock });
```

3. Generación de casos de prueba unitarios

Las pruebas unitarias cubren la lógica pura de app.js, especialmente el cálculo del estado de riego y la manipulación de datos de plantas.

Prompt utilizado:

```
Genera pruebas unitarias con Jest para las siguientes funciones de app.js: - calcularEstado(planta): devuelve 'ok', 'pronto' o 'ya' según los días transcurridos - calcularProgreso(planta): devuelve un porcentaje entre 0 y 100 - guardarPlantas(plantas): guarda el array en localStorage - cargarPlantas(): devuelve el array desde localStorage Incluye casos límite (array vacío, fechas futuras, frecuencia 0).
```

Tests generados (tests/unit/plantLogic.test.js):

```
import { calcularEstado, calcularProgreso, guardarPlantas, cargarPlantas } from '../src/app.js'; describe('calcularEstado', () => {  test('retorna ok cuando quedan más del 50% del intervalo', () => {    const hoy = new Date();    const ultimo = new Date(hoy);    ultimo.setDate(hoy.getDate() - 1);    const planta = { cada: 7, ultimo: ultimo.toISOString().split('T')[0] };    expect(calcularEstado(planta)).toBe('ok');  });  test('retorna pronto cuando queda menos del 50% pero no ha pasado el intervalo', () => {    const hoy = new Date();    const ultimo = new Date(hoy);    ultimo.setDate(hoy.getDate() - 5);    const planta = { cada: 7, ultimo: ultimo.toISOString().split('T')[0] };    expect(calcularEstado(planta)).toBe('pronto');  });  test('retorna ya cuando ha superado el intervalo', () => {    const hoy = new Date();    const ultimo = new Date(hoy);    ultimo.setDate(hoy.getDate() - 10);    const planta = { cada: 7, ultimo: ultimo.toISOString().split('T')[0] };  });
```

```
expect(calcularEstado(planta)).toBe('ya'); }); test('caso límite:
frecuencia 0 días', () => { const planta = { cada: 0, ultimo: '2025-01-
01' }; expect(calcularEstado(planta)).toBe('ya'); }); });
describe('guardarPlantas / cargarPlantas', () => { beforeEach(() =>
localStorage.clear()); test('guarda y recupera el array correctamente', () =>
{ const plantas = [{ id: 1, name: 'Monstera', tipo: 'interior', cada: 7,
ultimo: '2025-06-01' }]; guardarPlantas(plantas);
expect(cargarPlantas()).toEqual(plantas); }); test('retorna array vacío si
localStorage está vacío', () =>
{ expect(cargarPlantas()).toEqual([]); }); });
```

4. Generación de pruebas de integración

Las pruebas de integración verifican la interacción entre la lógica de negocio y el DOM simulado (jsdom), comprobando que añadir, editar y eliminar plantas actualiza correctamente la interfaz y el localStorage.

Prompt utilizado:

Genera pruebas de integración con Jest y jsdom para PlantCare que verifiquen: 1. Al llamar a añadirPlanta() con datos válidos, la tarjeta aparece en el DOM 2. Al llamar a eliminarPlanta(id), la tarjeta desaparece del DOM y se actualiza localStorage 3. El panel de resumen muestra el número correcto de plantas que necesitan riego Carga el HTML real desde index.html en cada test.

Tests generados (tests/integration/dom.test.js — fragmento):

```
import { readFileSync } from 'fs'; import { JSDOM } from 'jsdom'; let dom,
document, window; beforeEach(() => { const html =
readFileSync('./index.html', 'utf-8'); dom = new JSDOM(html, { runScripts:
'dangerously', resources: 'usable' }); document = dom.window.document;
window = dom.window; window.localStorage.clear(); }); test('añadir planta
muestra la tarjeta en el DOM', async () => { window.añadirPlanta({ id: 1,
name: 'Cactus', tipo: 'suculenta', cada: 14, ultimo: '2025-06-10' }); const
tarjetas = document.querySelectorAll('.plant-card');
expect(tarjetas.length).toBe(1);
expect(tarjetas[0].textContent).toContain('Cactus'); }); test('eliminar planta
la elimina del DOM', () => { window.añadirPlanta({ id: 2, name: 'Ficus', tipo:
'interior', cada: 7, ultimo: '2025-06-01' }); window.eliminarPlanta(2);
expect(document.querySelectorAll('.plant-card').length).toBe(0); });
```

5. Pruebas de interfaz (E2E con Playwright)

Para las pruebas de interfaz se utilizó Playwright, que permite automatizar el navegador real y verificar el comportamiento visual de la aplicación.

Prompt utilizado:

Genera pruebas E2E con Playwright para PlantCare que cubran el flujo completo: 1. Abrir la app, añadir una planta mediante el formulario y verificar que aparece en la lista 2. Hacer clic en el botón 'Regar' y verificar que el estado cambia 3. Navegar a la vista de calendario y verificar que el mes actual se muestra 4. Editar una planta y comprobar que los cambios se reflejan La app se sirve en http://localhost:3000.

Tests generados (tests/e2e/plantcare.spec.js — fragmento):

```
import { test, expect } from '@playwright/test'; test.describe('PlantCare E2E',
() => { test.beforeEach(async ({ page }) => { await
page.goto('http://localhost:3000'); // Limpiar localStorage antes de cada
test await page.evaluate(() => localStorage.clear()); await
page.reload(); }); test('añadir planta aparece en la lista', async
({ page }) => { await page.click('[data-testid="btn-add"]'); await
page.fill('#plant-name', 'Orquídea'); await page.selectOption('#plant-tipo',
'interior'); await page.fill('#plant-frequency', '5'); await
page.fill('#plant-last-watered', '2025-06-15'); await page.click('[data-
testid="btn-save"]'); await expect(page.locator('.plant-
card')).toContainText('Orquídea'); }); test('botón regar reinicia el estado
a ok', async ({ page }) => { // Setup: planta con riego vencido await
page.evaluate(() => { localStorage.setItem('plantas',
JSON.stringify([ { id: 1, name: 'Rosa', tipo: 'exterior', cada: 3,
ultimo: '2025-06-01' } ])); }); await page.reload(); await
page.click('[data-testid="btn-water-1"]'); await expect(page.locator('[data-
id="1"] .status-badge')).toHaveClass(/ok/); }); test('navegación a
calendario muestra mes actual', async ({ page }) => { await
page.click('[data-testid="btn-calendar"]'); const mes = new
Date().toLocaleString('es-ES', { month: 'long', year: 'numeric' }); await
expect(page.locator('.calendar-header')).toContainText(mes); }); });
```

6. Ejecución y verificación de resultados

Tras generar los tests, se ejecutó el entorno de pruebas y se pidió al asistente que corrigiera los fallos encontrados.

6.1 Ejecución de pruebas unitarias e integración

Comando ejecutado:

```
npm test -- --coverage
```

Resultado obtenido:

```
PASS tests/unit/plantLogic.test.js calcularEstado ✓ retorna ok cuando
quedan más del 50% del intervalo (3ms) ✓ retorna pronto cuando queda menos
del 50% pero no ha pasado el intervalo (1ms) ✓ retorna ya cuando ha
superado el intervalo (1ms) ✓ caso límite: frecuencia 0 días (2ms)
guardarPlantas / cargarPlantas ✓ guarda y recupera el array correctamente
(4ms) ✓ retorna array vacío si localStorage está vacío (1ms) PASS
tests/integration/dom.test.js ✓ añadir planta muestra la tarjeta en el DOM
(12ms) ✓ eliminar planta la elimina del DOM (8ms) Test Suites: 2 passed, 2
total Tests: 8 passed, 8 total Coverage: 78.4% statements | 71.2%
branches | 80.0% functions
```

6.2 Corrección de fallos con ayuda de la IA

Durante la ejecución inicial se detectó un fallo en el test de integración relacionado con la carga asíncrona del DOM. El siguiente prompt fue utilizado para corregirlo:

Prompt utilizado:

```
El test de integración falla con el error 'window.añadirPlanta is not a
function'. El HTML carga app.js como script externo. ¿Cómo puedo esperar a que
el script termina de cargar antes de ejecutar el test con jsdom?
```

El asistente propuso usar `waitFor` junto con un evento `'load'` del DOM, lo que resolvió el problema sin necesidad de modificar el código fuente de la aplicación.

6.3 Ejecución de pruebas E2E

Comando ejecutado:

```
npx playwright test --reporter=html
```

Resultado obtenido:

```
Running 3 tests using 1 worker    ✓ PlantCare E2E > añadir planta aparece en la
lista (1.8s)    ✓ PlantCare E2E > botón regar reinicia el estado a ok (2.1s)    ✓
PlantCare E2E > navegación a calendario muestra mes actual (1.4s)  3 passed
(5.3s) HTML report available at: playwright-report/index.html
```

7. Cobertura de pruebas

Módulo	Sentencias	Ramas	Funciones	Líneas
app.js (lógica)	82%	75%	86%	83%
DOM / render	68%	60%	72%	70%
localStorage utils	100%	100%	100%	100%
Global	78%	72%	80%	78%

La cobertura global del 78% supera el umbral recomendado del 70% para proyectos de esta envergadura. Las zonas con menor cobertura corresponden a ramas de manejo de errores en el renderizado del calendario, que requieren condiciones difíciles de reproducir de forma automatizada.

8. Estructura del repositorio y entregables

El código de pruebas se encuentra disponible en el mismo repositorio público de la Actividad 1, en la rama `testing`:

```
https://github.com/LadyMarina/PlantWaterApp/tree/testing
```

Estructura de carpetas:

```
PlantWaterApp/ |— index.html |— styles.css |— app.js |— package.json |—
jest.config.js |— playwright.config.js |— requirements.txt
dependencias Node listadas |— README.md |— instrucciones de
ejecución |— tests/ |— setup.js |— unit/ |—
plantLogic.test.js |— integration/ |— dom.test.js |— e2e/
|— plantcare.spec.js
```

README.md — instrucciones de ejecución:

```
# Ejecutar pruebas - PlantCare ## Requisitos - Node.js >= 18 - npm >= 9 ##
Instalación npm install npx playwright install chromium ## Pruebas unitarias e
integración npm test npm test -- --coverage # con informe de cobertura ##
Pruebas E2E npx serve . -l 3000 & # servidor local npx playwright test
npx playwright show-report # ver informe HTML
```

9. Análisis de resultados y conclusiones

9.1 Valoración del proceso asistido por IA

El uso del asistente de IA ha demostrado ser altamente eficiente para la generación del entorno de pruebas. Los prompts bien formulados permitieron obtener código funcional desde la primera iteración en la mayoría de los casos, reduciendo significativamente el tiempo de configuración.

Los aspectos donde la IA resultó más útil fueron:

- Generación de la configuración inicial de Jest y Playwright.
- Creación de mocks de localStorage sin dependencias externas.
- Propuesta de casos límite que el desarrollador podría pasar por alto (frecuencia 0, fechas futuras).
- Diagnóstico rápido del fallo de carga asíncrona en los tests de integración.

9.2 Limitaciones encontradas

Se identificaron las siguientes limitaciones en el proceso asistido:

- La IA generó inicialmente selectores CSS genéricos en los tests E2E, que fallaban porque el HTML no tenía atributos data-testid. Fue necesario un prompt adicional para adaptar los selectores.
- Para las pruebas del calendario, la IA propuso un enfoque demasiado acoplado al HTML interno. Se optó por simplificar el alcance de estos tests.
- El asistente no tiene acceso al código real del proyecto, por lo que algunos tests asumían nombres de función que diferían ligeramente de los reales, requiriendo ajustes menores.

9.3 Conclusión general

La actividad ha permitido verificar que la IA es una herramienta de gran valor en la fase de testing, no solo para generar código sino también para estructurar la estrategia de pruebas. La combinación de Jest para lógica unitaria y Playwright para pruebas E2E ha resultado ser una elección robusta para una aplicación Vanilla JS, y la cobertura obtenida del 78% garantiza una base sólida para el mantenimiento y la evolución del proyecto.

La clave del éxito ha residido en la calidad de los prompts: cuanto más específico y contextualizado es el prompt, más precisa y reutilizable es la respuesta del asistente.